Global Knowledge®

Angular Fundamentals

Observables

Peter Kassenaar –
info@kassenaar.com

WORLDWIDE LOCATIONS

BELGIUM CANADA COLOMBIA DENMARK EGYPT FRANCE IRELAND JAPAN KOREA MALAYSIA MEXICO NETHERLANDS NORWAY QATAR
SAUDI ARABIA SINGAPORE SPAIN SWEDEN UNITED ARAB EMIRATES UNITED KINGDOM UNITED STATES OF AMERICA

Async services met RxJS/Observables

Reactive programming with asynchronous streams

Async Services

- Statische data ophalen: *synchrone* actie
- Data ophalen uit backend: *asynchrone* actie
- Andere frameworks : `Promises`
- Angular: `Observables`

Bovendien in Angular: ReactiveX library `RxJS`

ReactiveX Introduction Docs Languages Resources Community

ReactiveX

An API for asynchronou
with observable stream

Choose your platform

http://reactivex.io/

ReactiveX Introduction Docs Languages Resources Community

Languages

- Java: RxJava
- JavaScript: RxJS
- C#: Rx.NET
- C#(Unity): UniRx
- Scala: RxScala
- Clojure: RxClojure
- C++: RxCpp
- Ruby: Rx.rb
- Python: RxPY
- Groovy: RxGroovy
- JRuby: RxJRuby
- Kotlin: RxKotlin
- Swift: RxSwift

ReactiveX for platforms and frameworks

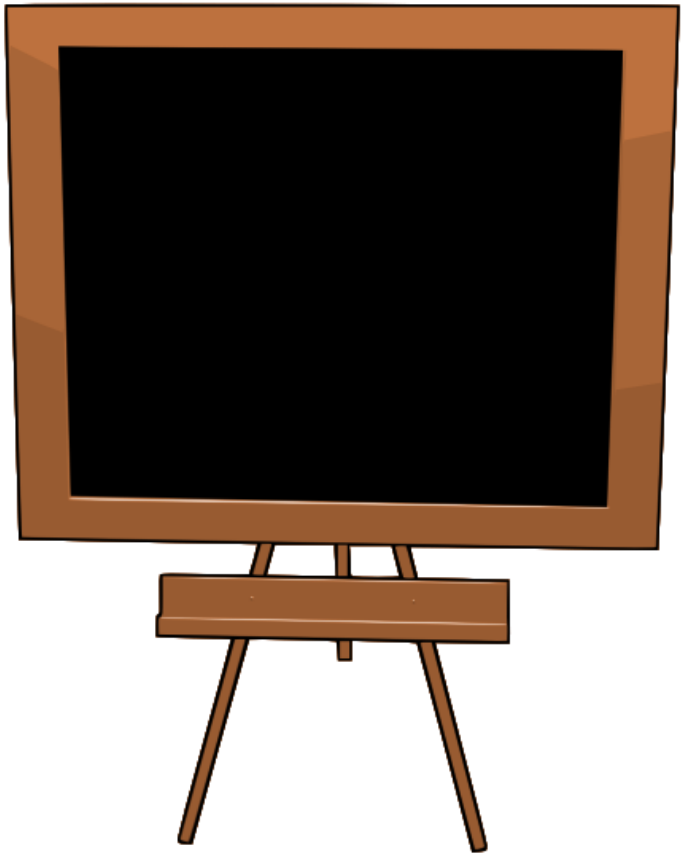
- RxNetty
- RxAndroid
- RxCocoa

DOCUMENTATION	LANGUAGES	RESOURCES	COMMUNITY
Observable	RxJava	Tutorials	GitHub
Operators	RxJS		Twitter
Single	Rx.NET		Others

The observable design pattern

"Understanding the observable stream"

Explanation



Why Observables?

We can do much more with observables than with promises.

With observables, we have a whole bunch of operators to pull from, which let us customize our streams in nearly any way we want.

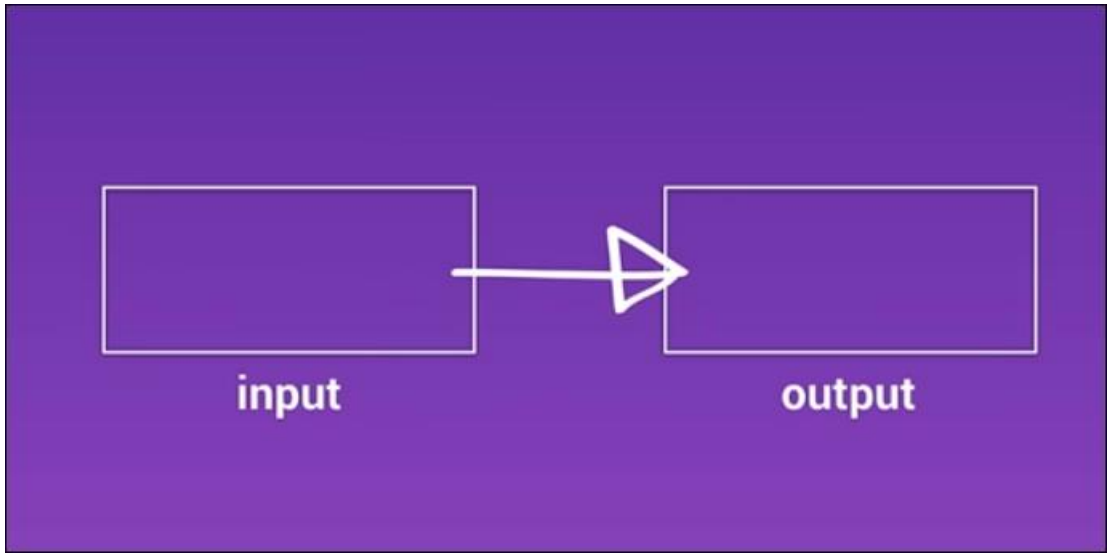
https://auth0.com/blog/2015/10/15/angular-2-series-part-3-using-http/

Observables en RxJs

- "Reactive Programming"
  - *"Reactive programming is programming with asynchronous data streams."*
  - https://gist.github.com/staltz/868e7e9bc2a7b8c1f754
- Observables hebben extra mogelijkheden ten opzichte van Promises
  - Mapping
  - Filtering
  - Combining
  - Cancel
  - Retry
  - …
- Gevolg: géén `.success()`, `.error()` en `.then()` chaining meer!

How do observables work

- First - *The Observable Stream*
- Later - all 10.000 operators…
- Traditionally:

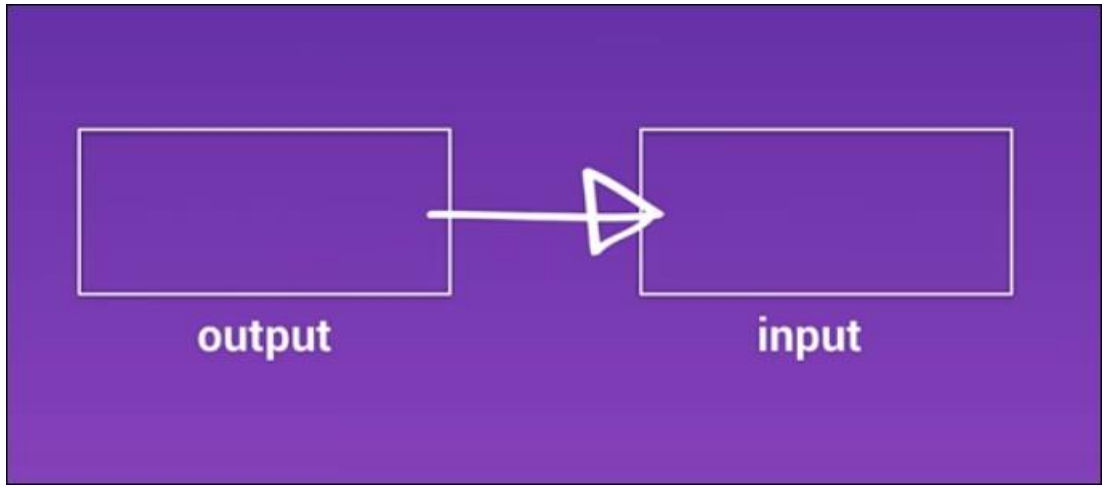


Go beast mode with realtime reactive interfaces in Angular 2 & Firebase | Lukas Ruebbelke

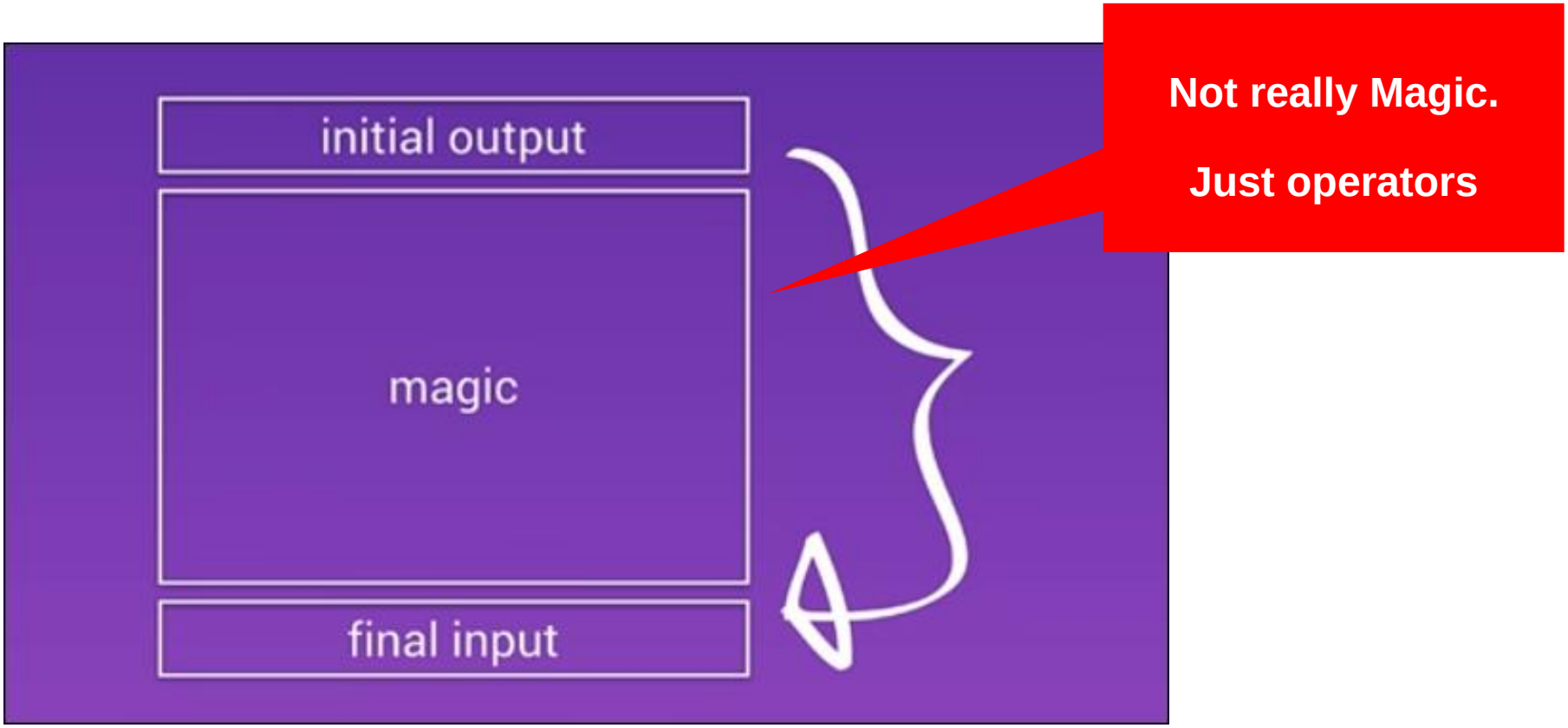
https://www.youtube.com/watch?v=5CTL7aqSvJU

https://youtu.be/5CTL7aqSvJU?t=4m31s

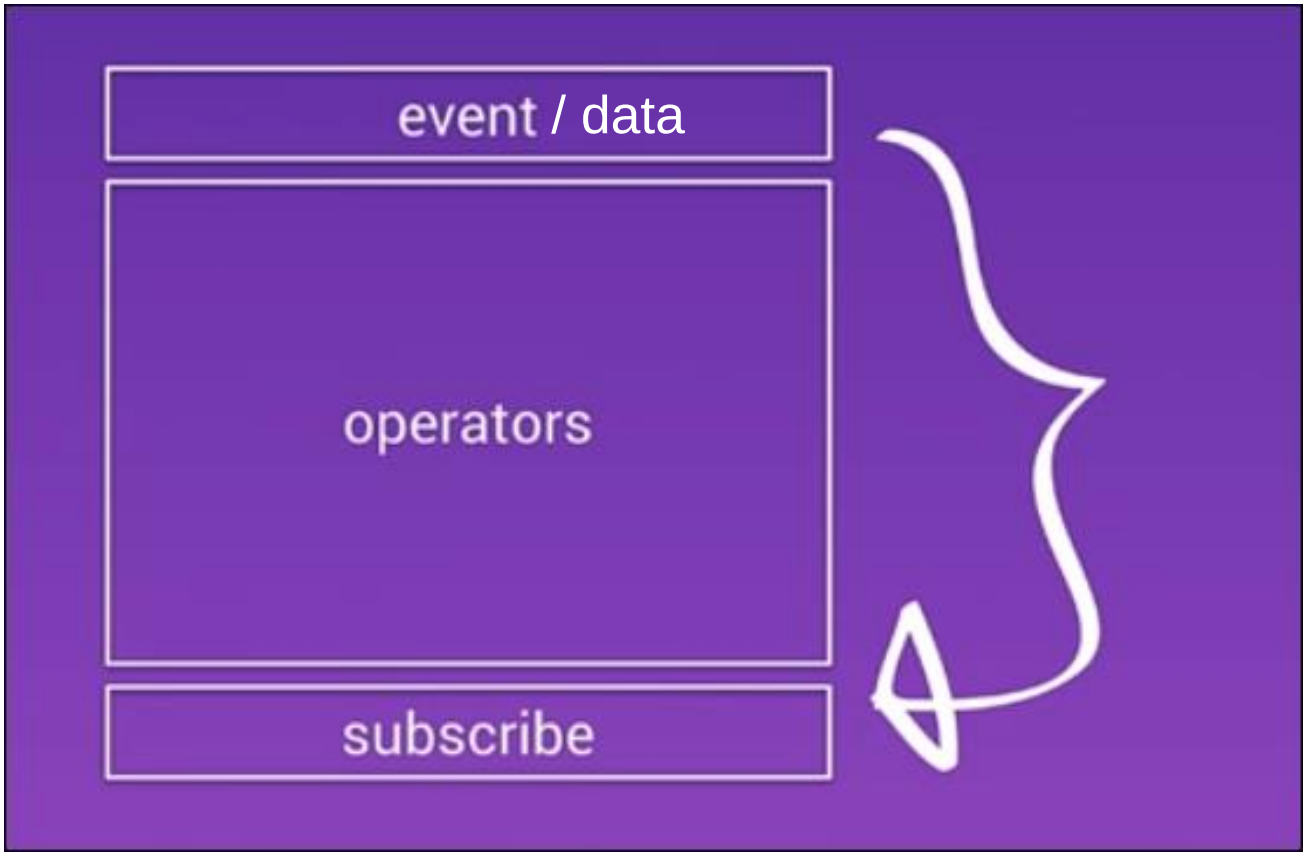
- With Observables -
  - a system, already outputting data,
  - Subscribe to that data
- "trade Output for Input"
- "Push vs. Pull"



"The observable sandwich"



Subscribe to events



In code:

```
this.http.get<City[]>('assets/data/cities.json')
    .pipe(
        delay(…),
        map(…)
    )
    .subscribe((result:City[]) => {
      //... Do something
    });
```

Initial Output

Optioneel:

operator(s)

Final Input

Ook: importeren `HttpClientModule` in `@ngModule`

- *// Angular Modules*

  …

- **import** {HttpClientModule} **from '@angular/common/http'**;

  // Module declaration

  @NgModule({

  imports : [BrowserModule, HttpClientModule],

  declarations: [AppComponent],

  bootstrap : [AppComponent],

  …

  })

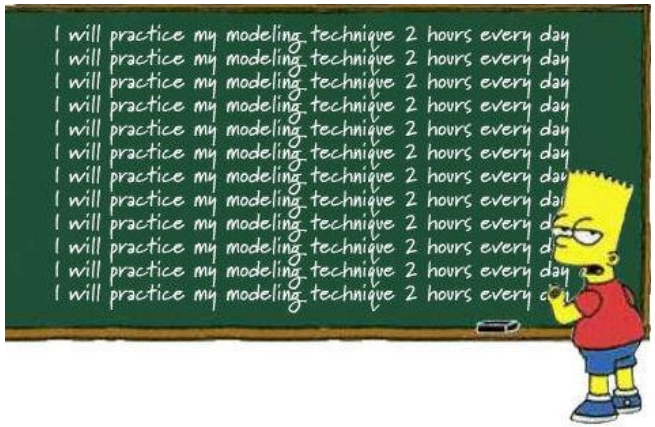
  export class AppModule {

  }

Oefening

- Bekijk het voorbeeld in `/201_services_http`
- Maak een eigen `.json`-bestand en importeer dit in je applicatie.
- Oefening `5b), 5c)`

Exercise….



Pipeable operators

- In RxJS 6.x en hoger: alle operators komen binnen de `.pipe()` functie
- De parameters van de pipe-functie zijn de operatoren!
- Ze worden met komma's van elkaar gescheiden

```
.pipe(
    delay(3000),
    retry(3),
    map(result => …),
    takeUntil(…condition…)
)
```

More on subscriptions

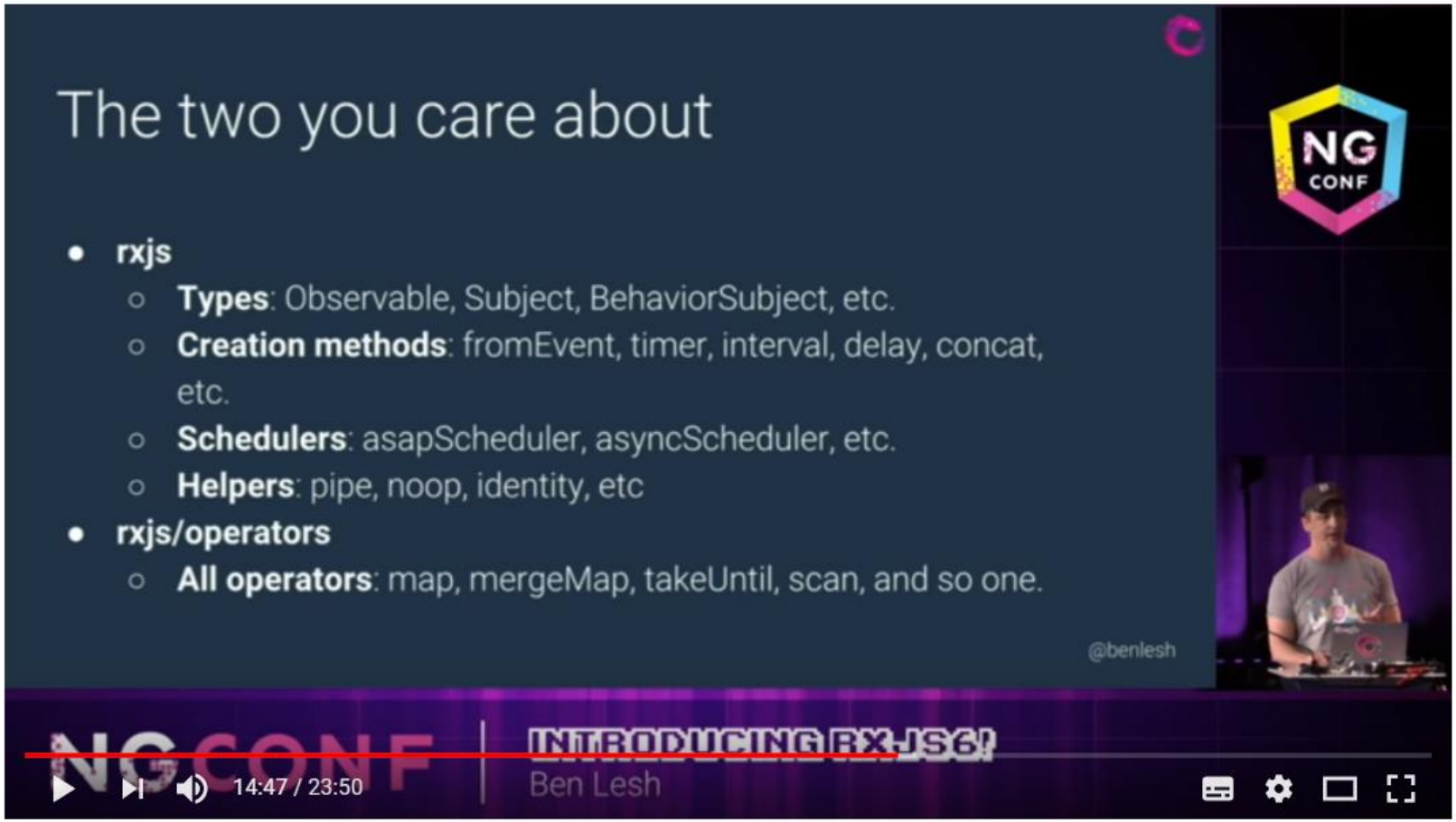
Using parameters inside the subscriber

Subscribe - only once per block!

- Three parameters:
  - success() / next()
  - error() – Optioneel!
  - complete() – Optioneel!

```
this.cityService.getCities()

   .subscribe(cityData => {
         this.cities = cityData;
      },
      err => console.log(err),
      ()=> console.log('Getting cities complete...')
   )
```

Ben Lesh on observables in RxJS 6.0



Introducing RxJS6! - Ben Lesh

https://www.youtube.com/watch?v=JCXZhe6KsxQ

Useful operators

- RxJS operators are (mostly) like Array operators
- Perform actions on a stream of objects
- Grouped by subject
  - Creation operators
  - Transforming
  - Filtering
  - Combining
  - Error Handling
  - Conditional and Boolean
  - Mathematical
  - …

https://www.learnrxjs.io/

learnrxjs.io/learn-rxjs/operators

Learn RxJS

Introduction

LEARN RXJS

- Operators
  - Combination
  - Conditional
  - Creation
  - Error Handling
  - Multicasting
  - Filtering
  - Transformation
  - Utility
  - Full Listing
- Subjects
- Recipes
- Concepts

Powered by GitBook

Operators

CONTENTS

Contents (By Operator Type)

Additional Resources

A complete list of RxJS operators with clear explanations, relevant resources, and executable examples.

Prefer a complete list in alphabetical order?

Contents (By Operator Type)

- Combination
  - combineAll
  - combineLatest ⭐
  - concat ⭐
  - concatAll
  - endWith
  - forkJoin
  - merge ⭐
  - mergeAll
  - pairwise
  - race
  - startWith ⭐
  - withLatestFrom ⭐
  - zip
- Conditional
  - defaultIfEmpty
  - every
  - iif
  - sequenceequal
- Creation

Async pipe

Automatische `.subscribe()` en `.unsubscribe()`

Async Pipe

- Bij `.subscribe()`, eigenlijk ook `.unsubscribe()` aanroepen.
  - Netjes!
  - Bij HTTP-requests niet beslist nodig, bij andere subscriptions wel, in verband met memory leaks.
- Niet meer zelf `.subscribe()` en `.unsubscribe()` aanroepen:
  - Gebruik `async` pipe van Angular

- In de component:

```
Cities$: Observable<City[]>; // Nu: Observable naar Type
…

ngOnInit() {
   // Call naar de service, retourneert Observable
   this.cities$ = this.cityService.getCities()
}
```

- In de view:

```
<li *ngFor="let city of cities$ | async”>
```

Werken met Live API's

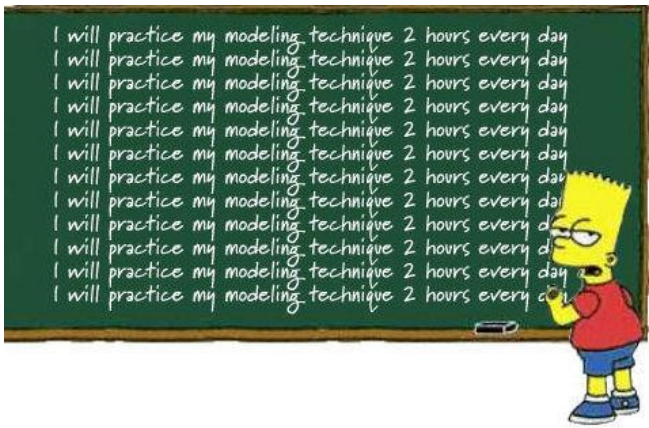
- MovieApp
- `examples\210-services-live`

Voorbeeld API's

- https://pokeapi.co/ - Pokemon API
- http://openweathermap.org/API (weerbericht)
- https://jsonplaceholder.typicode.com/ random users, posts, photos
- http://ergast.com/mrd/ - Ergast Motor (F1) API
- http://www.omdbapi.com/ - Open Movie Database
- https://swapi.dev/ - Star Wars API
- Zie ook `JavaScript APIs.txt` met meer voorbeelden

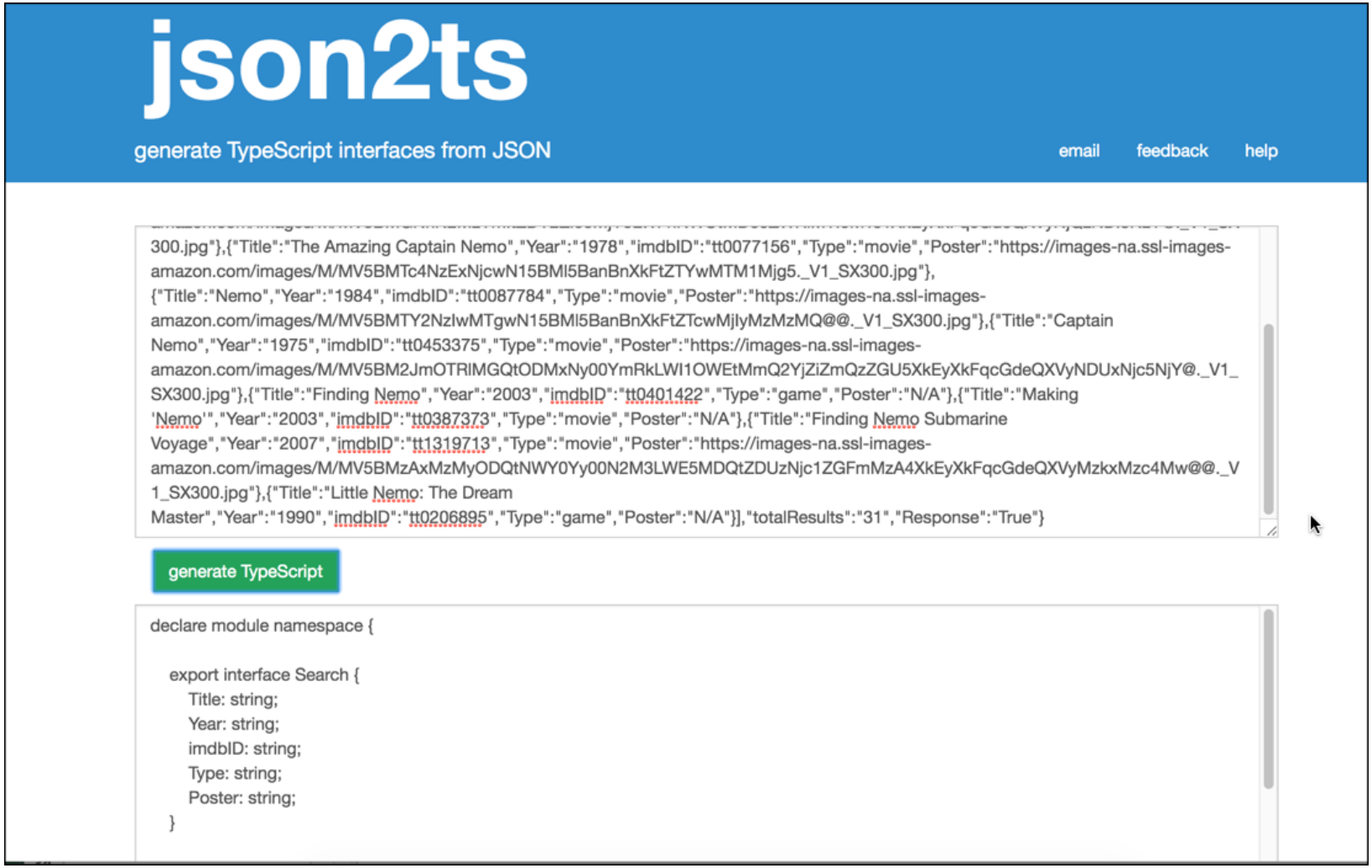
Workshop

- Pick one of your own projects, or see for instance:
  - `../210-services-live`
- Create a small application using one of the API's in the file `JavaScript API's.txt`, using RxJS-calls, for example
  - Pokemon API
  - Kenteken API
  - OpenWeatherMap API
  - …
- Exercise : 5e)



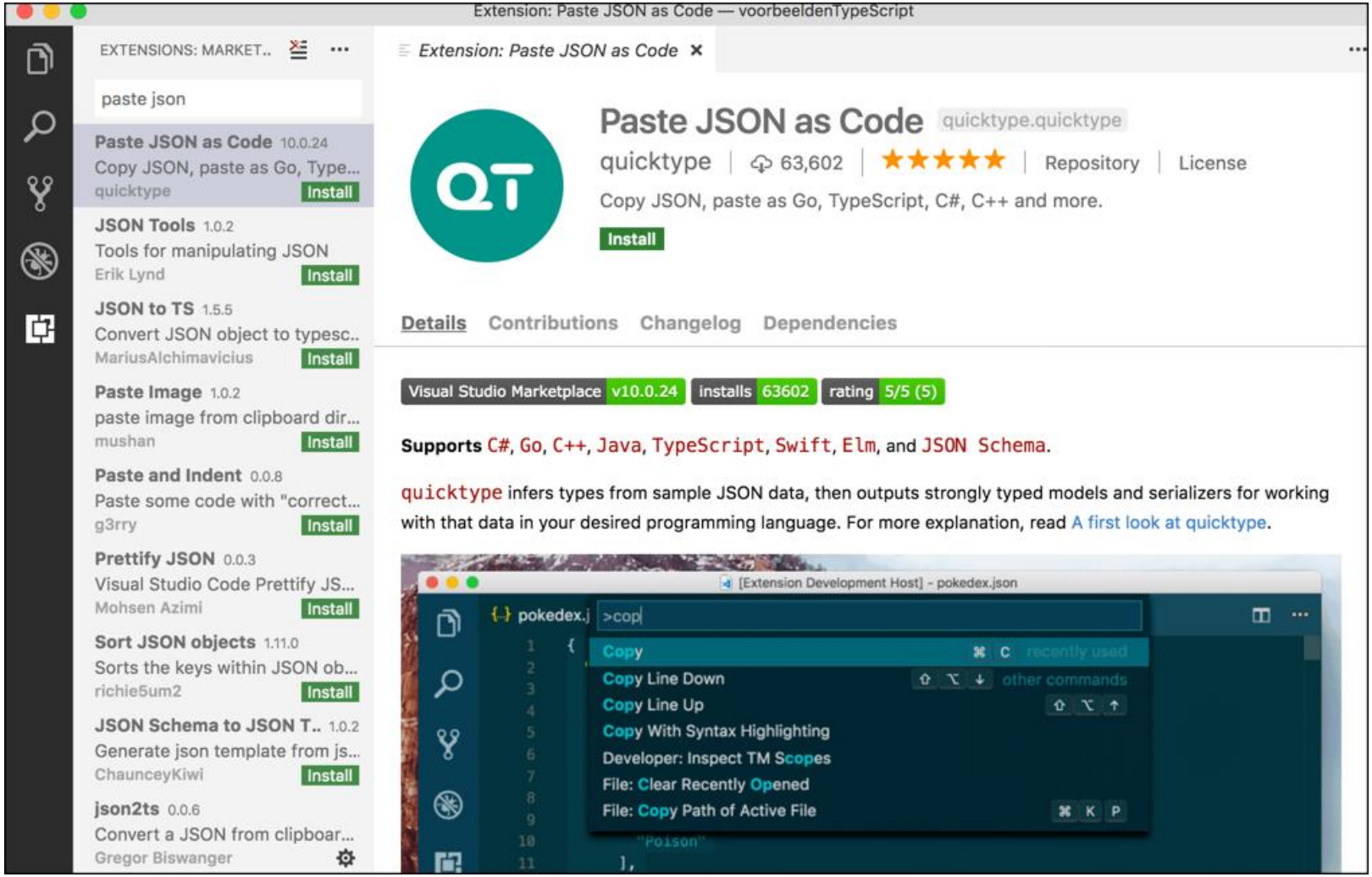
More info on observables

Online JSON to TypeScript converter



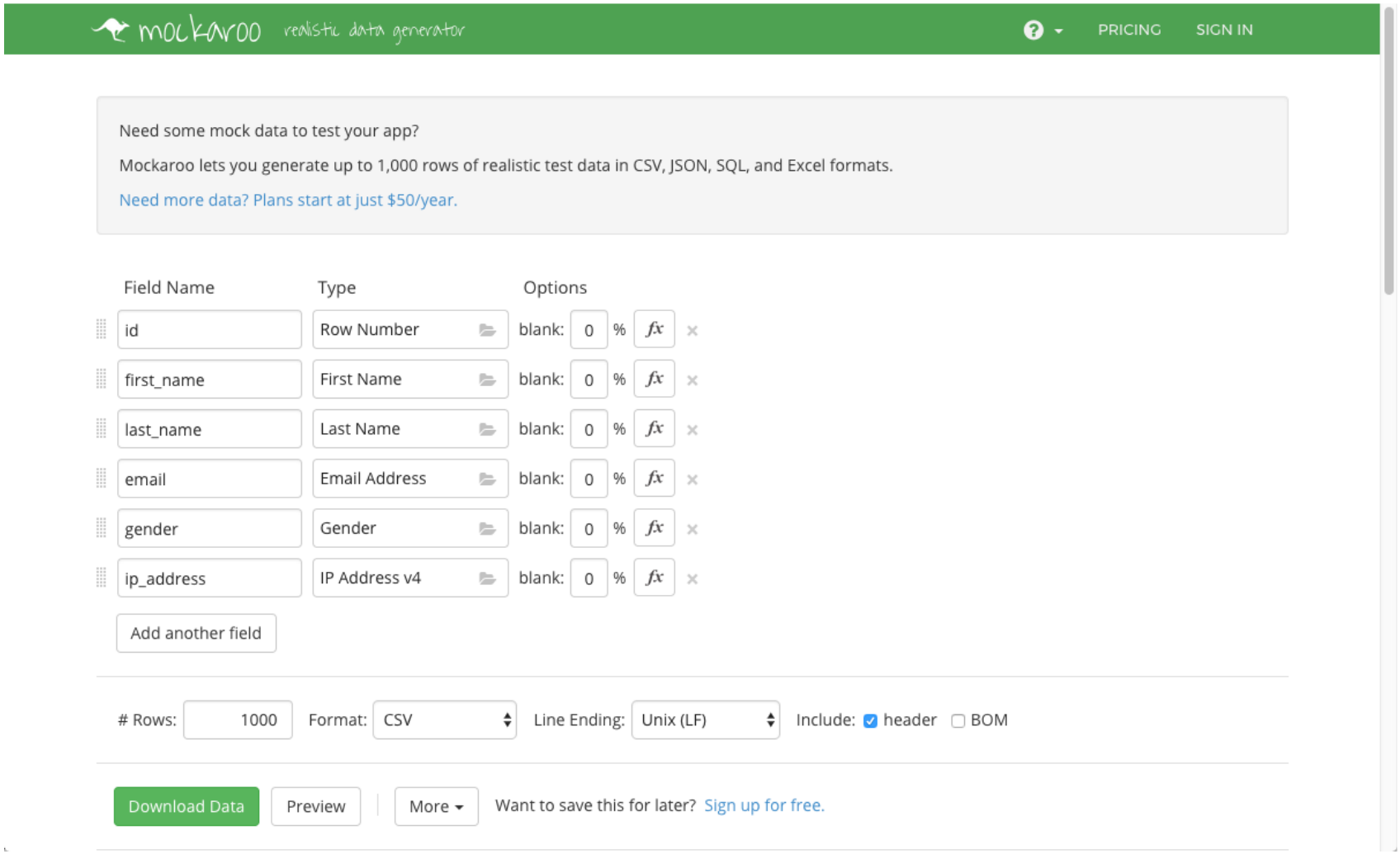
http://json2ts.com/

In VS Code? Use this extension!



https://marketplace.visualstudio.com/items?itemName=quicktype.quicktype

Data Mocken - Mockaroo



http://mockaroo.com/

Official documentation…

RxJS Home Manual Reference Source Test dark theme light theme

- AsyncSubject
- BehaviorSubject
- Notification
- Observable
- ReplaySubject
- Scheduler
- AnonymousSubject
- Subject
- SubjectSubscriber
- Subscriber
- Subscription
- ObservableInput
- Observer
- SubscribableOrPromise
- TeardownLogic
- Rx.Scheduler
- Rx.Symbol

observable
- ConnectableObservable

observable/dom
- AjaxError
- AjaxResponse
- AjaxTimeoutError
- AjaxRequest

operator
- GroupedObservable

scheduler
- Action

```
import {Observable} from '@reactivex/rxjs/es6/Observable.js'
```

public class | source

Observable

Direct Subclass:
ConnectableObservable, GroupedObservable, Subject

Indirect Subclass:
AnonymousSubject, AsyncSubject, BehaviorSubject, es6/operator/windowTime.js~CountedSubject, ReplaySubject

A representation of any set of values over any amount of time. This the most basic building block of RxJS.

Test:
Observable
Observable.create
Observable.lift

Static Method Summary

Static Public Methods	
public static	bindCallback(func: function, selector: function, scheduler: Scheduler): function(...params: *): Observable Converts a callback API to a function that returns an Observable.
public static	bindNodeCallback(func: function, selector: function, scheduler: Scheduler): function(...params: *): Observable

Generated by ESDoc(0.4.8)

http://reactivex.io/rxjs/class/es6/Observable.js~Observable.html

https://www.learnrxjs.io/

Learn RxJS

Introduction

LEARN RXJS

Operators
- Combination
- Conditional
- Creation
- Error Handling
- Multicasting
- Filtering
- Transformation
- Utility
- Full Listing

Subjects

Recipes

Concepts

Powered by GitBook

Operators

CONTENTS

Contents (By Operator Type)

Additional Resources

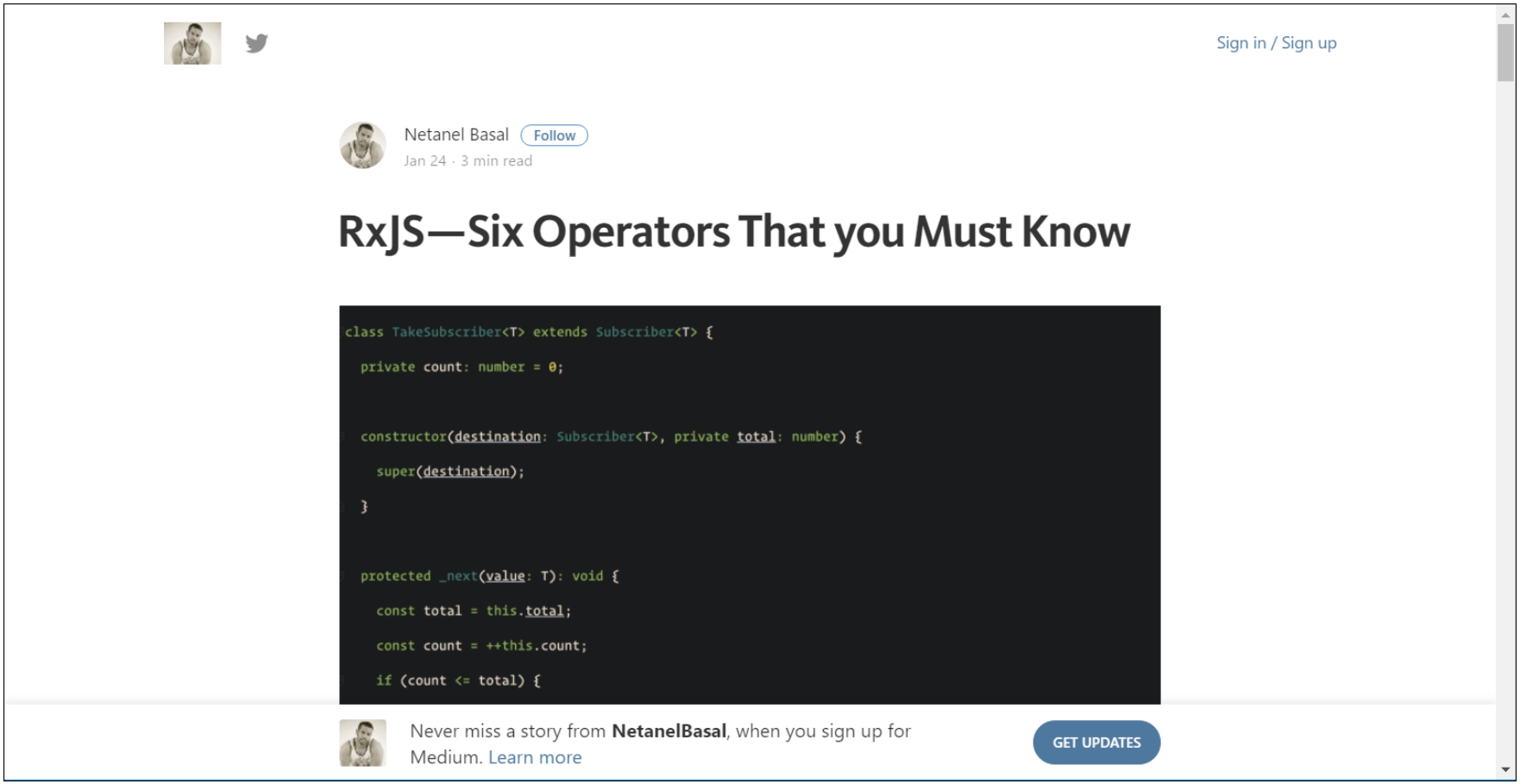
A complete list of RxJS operators with clear explanations, relevant resources, and executable examples.

Prefer a complete list in alphabetical order?

Contents (By Operator Type)

- Combination
  - combineAll
  - combineLatest ★
  - concat ★
  - concatAll
  - endWith
  - forkJoin
  - merge ★
  - mergeAll
  - pairwise
  - race
  - startWith ★
  - withLatestFrom ★
  - zip
- Conditional
  - defaultIfEmpty
  - every
  - iif
  - sequenceequal
- Creation

Article - 6 Operators you must know



Sign in / Sign up

Netanel Basal Follow

Jan 24 · 3 min read

RxJS—Six Operators That you Must Know

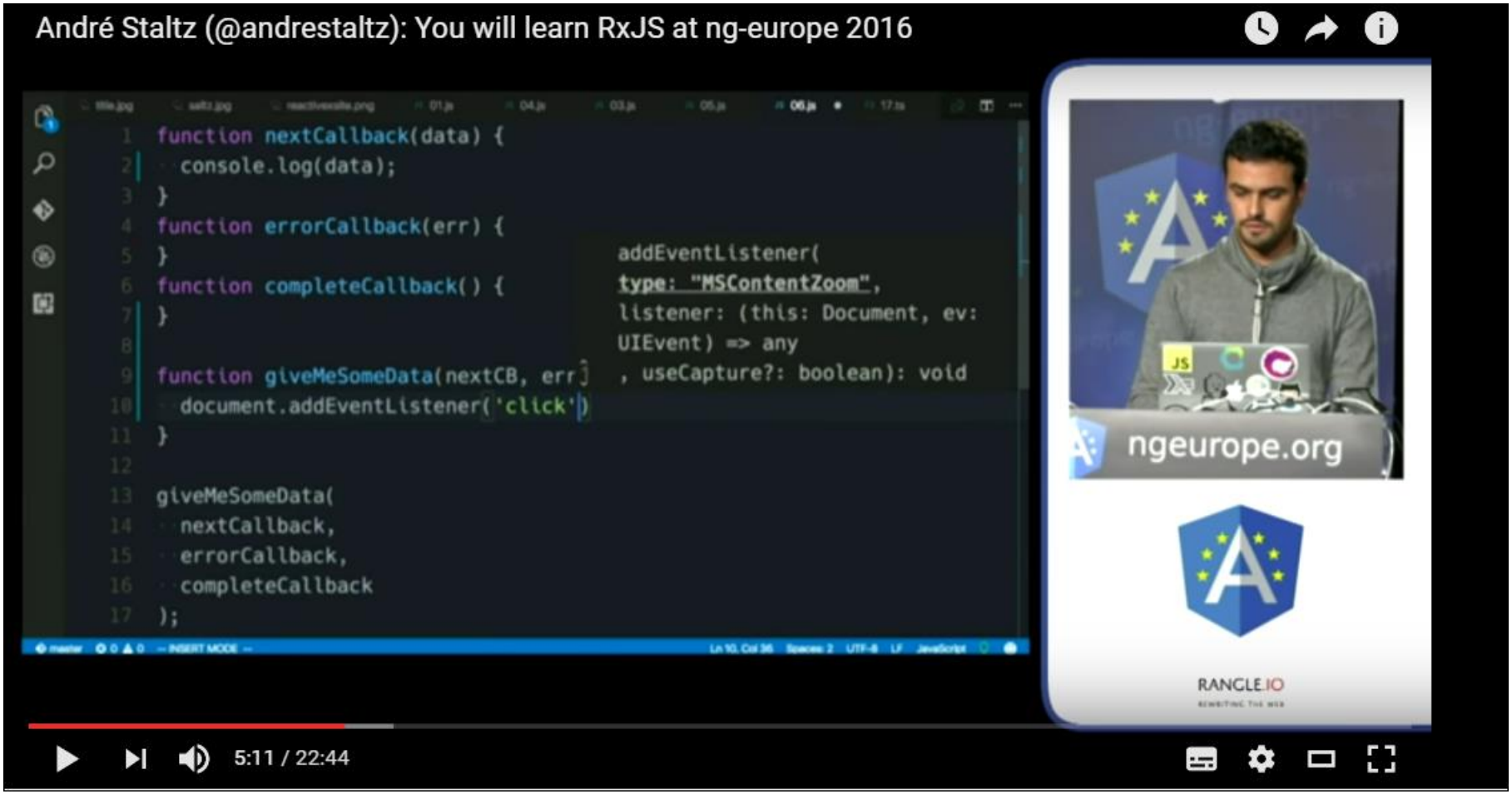
```
class TakeSubscriber<T> extends Subscriber<T> {
  private count: number = 0;

  constructor(destination: Subscriber<T>, private total: number) {
    super(destination);
  }

  protected _next(value: T): void {
    const total = this.total;
    const count = ++this.count;
    if (count <= total) {
```

Never miss a story from **NetanelBasal**, when you sign up for Medium. Learn more

GET UPDATES

https://netbasal.com/rxjs-six-operators-that-you-must-know-5ed3b6e238a0#.11of73aox

Creating Observables from scratch - André Staltz



https://www.youtube.com/watch?v=uQ1zhJHclvs

GitHub Gist Search... All gists GitHub New gist

staltz / introrx.md
Last active an hour ago

Star 10,812 Fork 1203

Code Revisions 259 Stars 10812 Forks 1203

Embed <script src="https://gist. Download ZIP

The introduction to Reactive Programming you've been missing

introrx.md Raw

The introduction to Reactive Programming you've been missing

(by @andrestaltz)

This tutorial as a series of videos

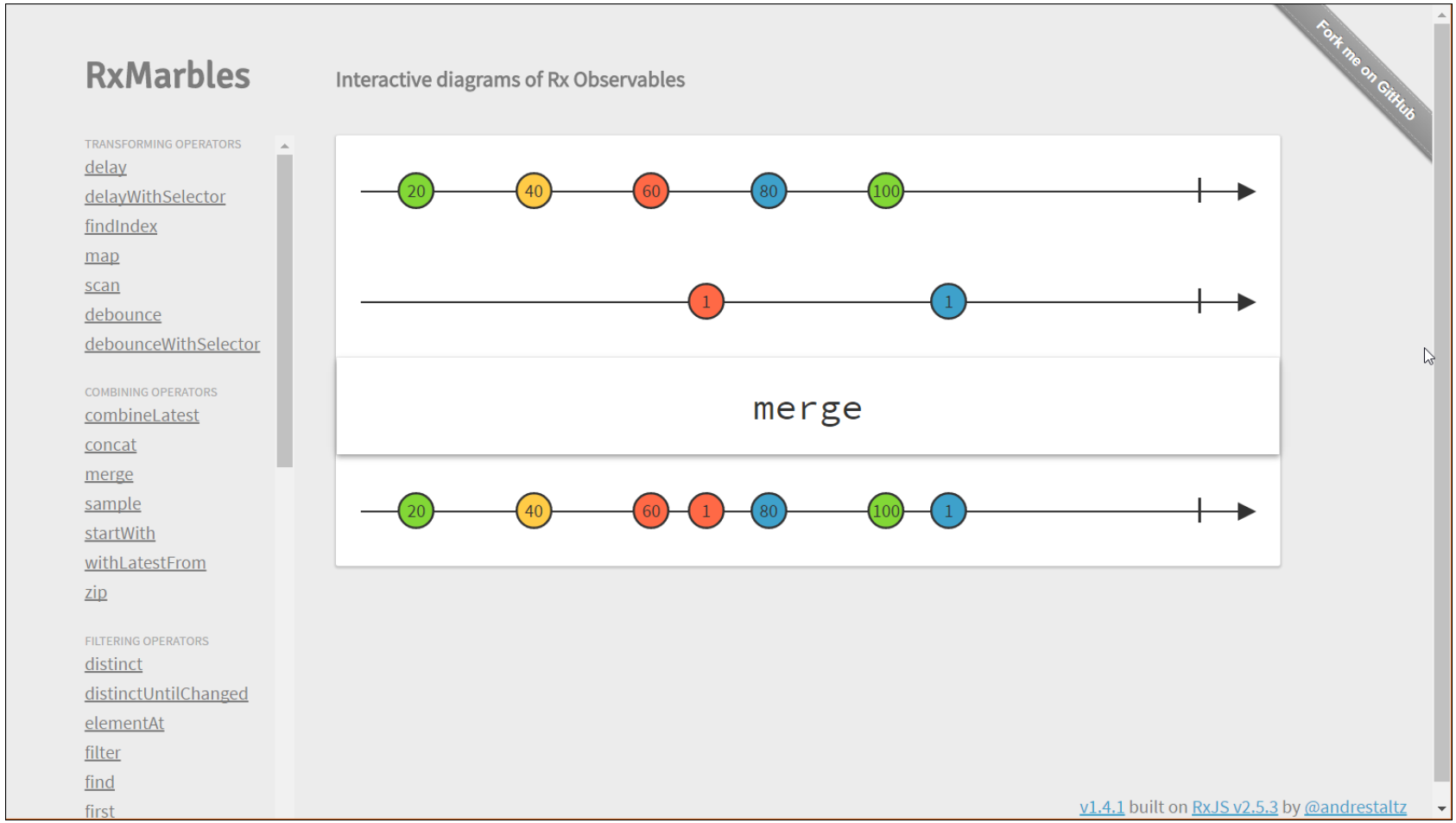
If you prefer to watch video tutorials with live-coding, then check out this series I recorded with the same contents as in this article: Egghead.io - Introduction to Reactive Programming.

So you're curious in learning this new thing called Reactive Programming, particularly its variant comprising of Rx, Bacon.js, RAC, and others.

Learning it is hard, even harder by the lack of good material. When I started, I tried looking for tutorials. I found only a handful of practical guides, but they just scratched the surface and never tackled the challenge of building the whole architecture

https://gist.github.com/staltz/868e7e9bc2a7b8c1f754

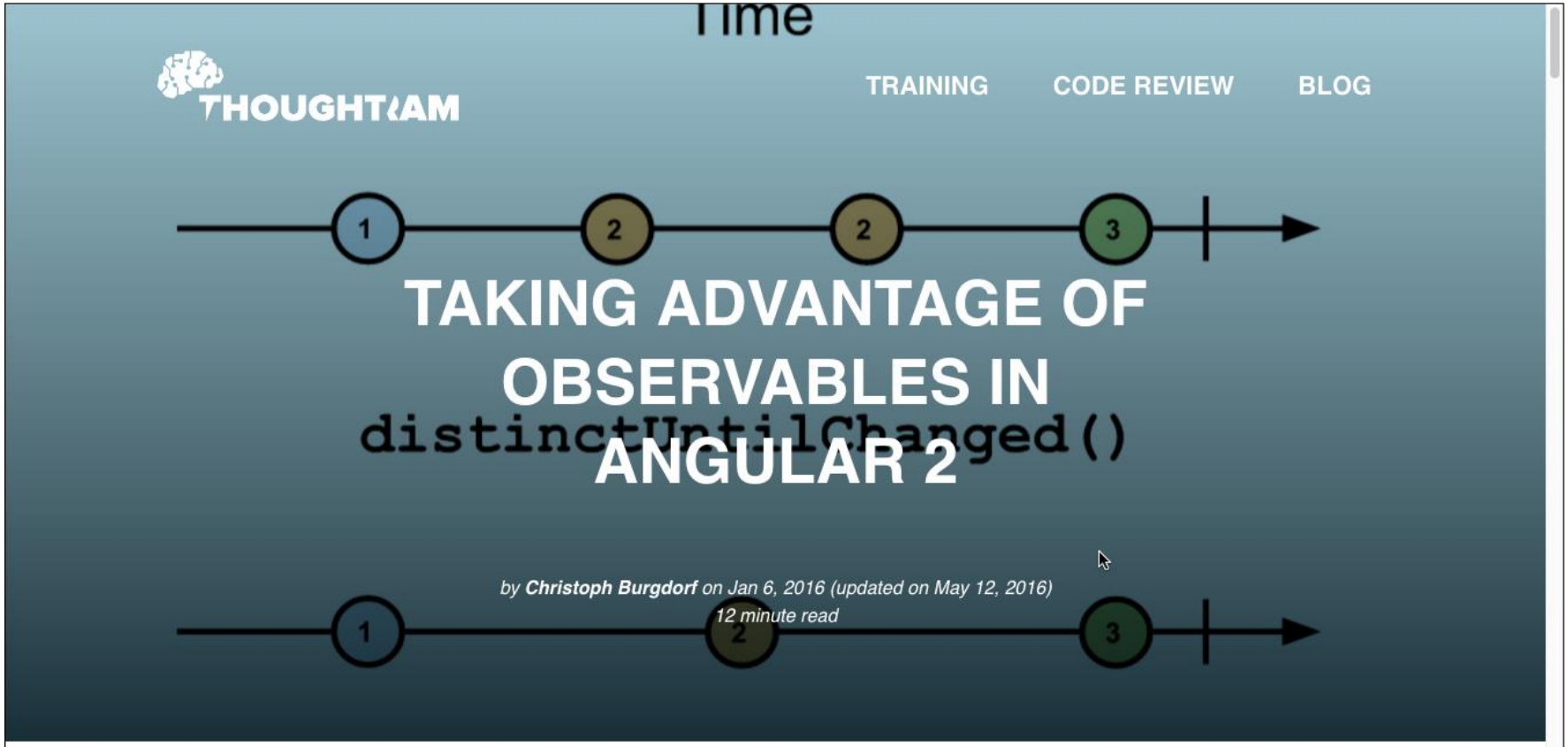
Also by Andre Stalz - RxMarbles



http://rxmarbles.com/

Dan Wahlin on Modules and Observables

https://www.youtube.com/watch?v=YxK4UW4UfCk



Some people seem to be confused why Angular 2 seems to favor the Observable abstraction over the Promise abstraction when it comes to dealing with async behavior.

There are pretty good resources about the difference between Observables and Promises already out there. I especially like to highlight this free 7 minutes video by Ben Lesh on egghead.io. Technically there are a couple of obvious differences like the *disposability* and *lazyness* of Observables. In this article we like to focus on some practical advantages that

http://blog.thoughtram.io/angular/2016/01/06/taking-advantage-of-observables-in-angular2.html

Een collectie observables ophalen

https://blog.angularindepth.com/practical-rxjs-in-the-wild-requests-with-concatmap-vs-mergemap-vs-forkjoin-11e5b2efe293